

Homework 5

Taylor Evans

2026-09-29

Set up

```
library(palmerpenguins)
```

```
##  
## Attaching package: 'palmerpenguins'  
  
## The following objects are masked from 'package:datasets':  
##  
##   penguins, penguins_raw
```

```
head(penguins)
```

```
## # A tibble: 6 x 8  
##   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  
##   <fct>   <fct>         <dbl>         <dbl>           <int>         <int>  
## 1 Adelie  Torgersen         39.1          18.7            181          3750  
## 2 Adelie  Torgersen         39.5          17.4            186          3800  
## 3 Adelie  Torgersen         40.3          18             195          3250  
## 4 Adelie  Torgersen         NA           NA              NA           NA  
## 5 Adelie  Torgersen         36.7          19.3            193          3450  
## 6 Adelie  Torgersen         39.3          20.6            190          3650  
## # i 2 more variables: sex <fct>, year <int>
```

- `library(palmerpenguins)` loads the package, which makes the dataset available as `penguins`.
- `head(penguins)` shows the first six rows so we can inspect the variables

Task 1: Calculate Summary Statistics

Objective: Use the 'apply' function to calculate column-wise means for the numeric variables in the dataset.

```
numeric_means <- apply(penguins[, sapply(penguins, is.numeric)], 2, mean, na.rm = TRUE)  
print(numeric_means)
```

```
##   bill_length_mm   bill_depth_mm flipper_length_mm   body_mass_g  
##      43.92193      17.15117      200.91520      4201.75439  
##      year  
##    2008.02907
```

The `apply()` function evaluates a function over the margins of an array or matrix. Here it is used to compute the mean of each numeric column in the dataset.

- `penguins[, sapply(penguins, is.numeric)]` selects only the numeric columns from the dataset. `sapply` checks each column with `is.numeric` and returns TRUE or FALSE, and that result keeps only the numeric columns.
- The 2 is the MARGIN argument. It specifies that the function should be applied to columns (use 1 for rows). MARGIN 2 preserves the columns and collapses the rows.
- `mean, na.rm = TRUE` calculates the mean for each column, ignoring missing (NA) values. `na.rm = TRUE` is passed to `mean` through the `...` argument.

Task 2: Count Penguins by Species

Objective: Use the 'tapply' function to count the number of penguins for each species.

```
species_counts <- tapply(penguins$species, penguins$species, length)
print(species_counts)
```

```
##      Adelie Chinstrap      Gentoo
##      152         68       124
```

The `tapply()` function is short for table apply, and it applies a function over subsets of a vector. Here it is used to count the number of penguins per species.

- The first `penguins$species` is the vector that is split into groups based on unique species values.
- The second `penguins$species` specifies the factor to group by.
- `length` counts the number of occurrences in each group.

Task 3: Analyze Bill Length by Species

Objective: Use the 'lapply' function to calculate the mean bill length for each species.

```
bill_length_means <- lapply(split(penguins$bill_length_mm, penguins$species),
                             mean, na.rm = TRUE)
print(bill_length_means)
```

```
## $Adelie
## [1] 38.79139
##
## $Chinstrap
## [1] 48.83382
##
## $Gentoo
## [1] 47.50488
```

The `lapply()` function loops over a list of objects and applies a function to every element of that list. Here it is used to calculate the mean bill length for each species.

- `split(penguins$bill_length_mm, penguins$species)` splits the bill lengths into separate lists based on species.
- `lapply(..., mean, na.rm = TRUE)` applies the `mean()` function to each species' list.
- `na.rm = TRUE` ensures missing values (NA) are ignored.

Task 4: Create a Summary Table

Objective: Use the `sapply` function to create a summary table with the mean and standard deviation for each numeric variable.

```
summary_table <- sapply(penguins[, sapply(penguins, is.numeric)],  
                        function(x) c(mean = mean(x, na.rm = TRUE),  
                                      sd = sd(x, na.rm = TRUE)))  
print(summary_table)
```

```
##      bill_length_mm bill_depth_mm flipper_length_mm body_mass_g      year  
## mean      43.921930      17.151170      200.91520    4201.7544 2008.0290698  
## sd        5.459584       1.974793       14.06171     801.9545  0.8183559
```

The `sapply()` function is a variant of `lapply` that simplifies the result if possible. Here it is used to compute both the mean and standard deviation for each numeric variable.

- `penguins[, sapply(penguins, is.numeric)]` selects only the numeric columns.
- The anonymous function `function(x) c(mean = mean(x, na.rm = TRUE), sd = sd(x, na.rm = TRUE))` calculates both the mean and the standard deviation. An anonymous function is a function that does not have a name, it is written on the fly inside the loop function.